Reflective Report

Personal Finance Management System

Techniques, Tools, and Technologies

1. Introduction

The development of the Personal Finance Management System (PFMS) provided a meaningful opportunity to engage with a broad range of database concepts in a practical and applied context. Working with a schema of 16 interrelated entities and over 30,000 synthetic records, the team developed a deeper appreciation for how relational databases function under realistic conditions, and how the decisions made at the design stage ripple through every subsequent phase of development.

2. Database Design and Schema Management

One of the most significant learnings came from the process of designing and refining the relational schema. Creating tables such as Transactions, Budget, Investment, and Loan required careful thought about normalization, foreign key relationships, and constraint enforcement. The team learned that constraints such as CHECK, NOT NULL, and UNIQUE are not merely formalities — they actively protect the integrity of data at the database level, independent of any application logic sitting above it.

A recurring challenge was managing derived attributes. Fields such as consumed_amount in Budget and remaining loan balance in Loan could not simply be stored as static values without risking inconsistency. This led to an important architectural decision: where possible, derive values dynamically using SUM() queries over the Transactions table rather than maintaining redundant stored values. This trade-off between storage efficiency and query simplicity was one of the most instructive discussions the team had throughout the project.

3. Triggers and Procedural Logic

Working with PL/pgSQL triggers was one of the most technically demanding but rewarding aspects of the project. The team implemented five core triggers covering automatic account creation on user registration, balance updates on transaction insertion, insufficient balance prevention, loan repayment tracking, and financial goal completion detection. Through this process, several non-obvious lessons emerged.

The most important lesson was understanding trigger execution order. When multiple triggers fire on the same event — such as AFTER INSERT ON Transactions — PostgreSQL fires them in alphabetical order by trigger name. This caused a subtle but critical bug: the budget alert trigger was reading consumed_amount before the budget update trigger had modified it, resulting in notifications never being sent. The fix required either merging both triggers into a single function or carefully naming triggers to control

their firing sequence. This experience demonstrated that trigger design requires thinking beyond individual functions in isolation.

The team also learned the distinction between BEFORE and AFTER triggers, and how RETURN NEW versus RETURN OLD affects row-level operations. Using BEFORE INSERT for balance validation allowed the team to block invalid transactions before any data was written, which proved far cleaner than attempting rollbacks after the fact.

4. Functions and Query Design, Tools

Writing reusable SQL functions such as get_net_cashflow, get_loan_remaining, get_total_balance, and get_budget_usage reinforced the value of encapsulating logic at the database level. These functions made it possible to perform complex multi-table calculations through a single function call, reducing the complexity that would otherwise need to be handled at the application layer. The team also became comfortable using DECLARE blocks, INTO clauses, and NULL handling within PL/pgSQL — skills that required practice to apply correctly and confidently.

The project was built on PostgreSQL, and the team worked extensively with pgAdmin as the primary interface for query execution, trigger testing, and schema inspection. One practical difficulty encountered was importing large CSV datasets generated with Python's Faker library. The team discovered that PostgreSQL's server-side COPY command cannot access client-side file paths, and that the correct approach is to use either pgAdmin's built-in Import/Export tool or the psql client-side \copy command. Resolving this issue required understanding the distinction between server-side and client-side execution contexts — a concept that is easy to overlook but has significant practical consequences